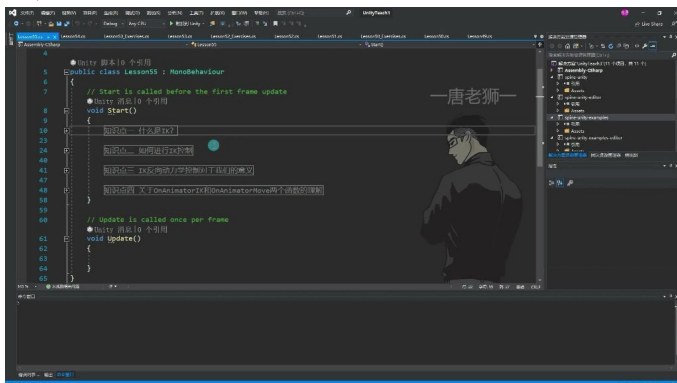


一、相关动画IK控制 00:04

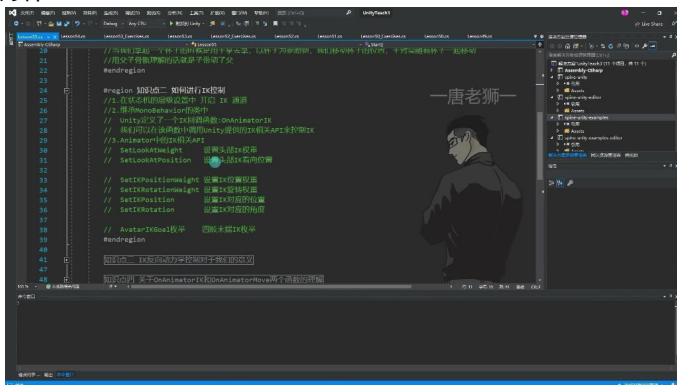
1. 什么是IK 00:35



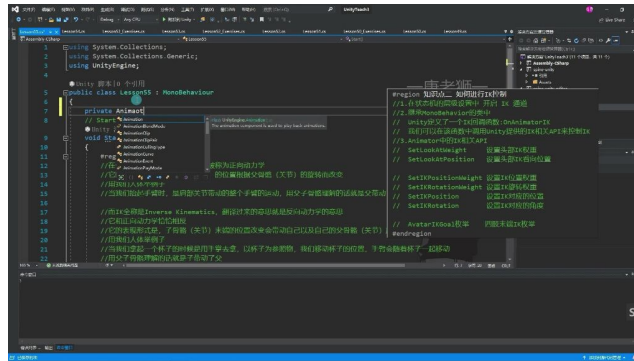
-
- **正向动力学(FK):** 在骨骼动画中构建骨骼的方法，子骨骼关节位置根据父骨骼关节旋转而改变。例如抬起手臂时，肩部关节带动整个手臂运动，体现"父带动子"的关系。
- **反向动力学(ik):** 全称Inverse Kinematics，与FK相反，子骨骼末端位置改变会带动自身及父骨骼旋转。例如拿杯子时手掌移动带动手臂运动，体现"子带动父"的关系。
- **应用场景:** 主要用于3D动画控制，相比2D动画，3D动画的IK控制不需要制作动画，只需通过代码控制移动旋转。

2. 如何进行IK控制 02:06

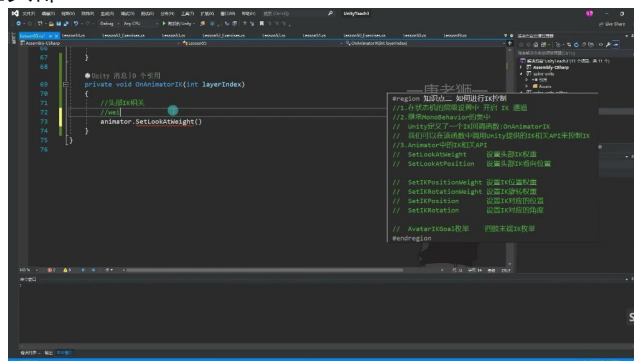
1) 制作IK 02:42



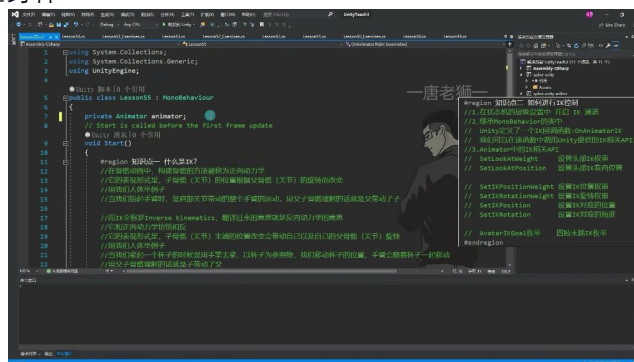
-
- **基本步骤:**
 - 在状态机层级设置中开启IK通道
 - 继承MonoBehavior的类中使用OnAnimatorIK回调函数
 - 调用Animator提供的IK相关API
- **准备工作:**
 - 创建动画状态机并添加待机动画
 - 必须开启层级设置的IK Pass选项，否则IK代码不会生效
- 2) OnAnimatorIK函数 04:58
- 声明成员变量 06:20



-
- **Animator变量:** 需要声明private Animator animator成员变量来获取Animator组件
- **目标点变量:** 使用public Transform声明控制点的位置变量
- 得到Animator组件 06:30
 - **获取方式:** 在Start()函数中通过GetComponent获取对象自身的Animator组件
- 设置头部IK 06:42

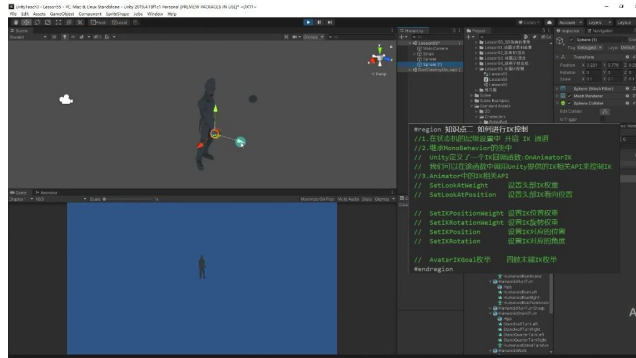


-
- **SetLookAtWeight参数:**
 - weight: 全局权重(0~1)
 - bodyWeight: 身体权重(0~1)
 - headWeight: 头部权重(0~1)
 - eyesWeight: 眼睛权重(0~1)
 - clampWeight: 运动限制(0不受限, 1完全固定, 0.5移动范围一半)
- **使用方法:**
 - 先设置权重再设置位置
 - 示例: animator.SetLookAtWeight(1, 0.5f, 0.5f)
 - 示例: animator.SetLookAtPosition(target.position)
- 设置身体IK 10:40



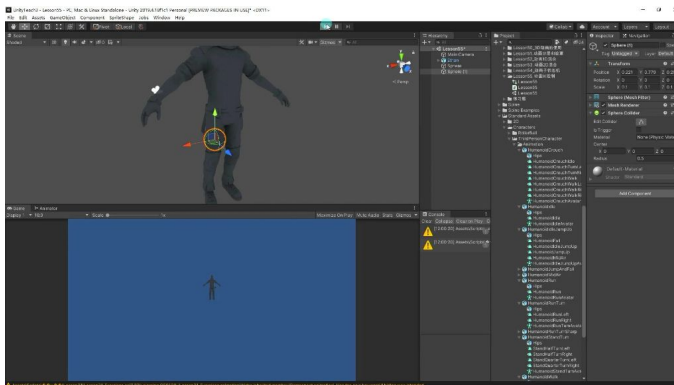
-
- **四肢控制API:**
 - SetIKPositionWeight: 设置位置权重
 - SetIKRotationWeight: 设置旋转权重
 - SetIKPosition: 设置目标位置

- SetIKRotation: 设置目标旋转
 - AvatarIKGoal枚举:
 - RightHand/LeftHand: 左右手
 - RightFoot/LeftFoot: 左右脚
- 设置四肢IK 14:22

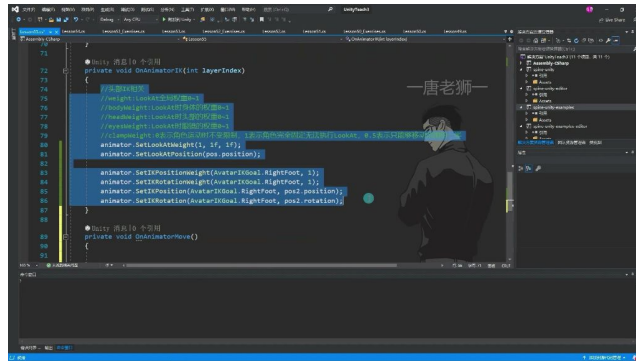


- 使用流程:
 - 设置权重(通常设为1)
 - 设置位置和旋转
 - 通过目标点Transform控制肢体运动
 - 注意事项:
 - 位置和旋转权重需要同时设置
 - 肢体运动范围受动画导入设置中的Rig页签限制
 - 可以单独控制头部、身体或四肢的IK效果

3. IK反向动力学控制对于我们的意义 21:50

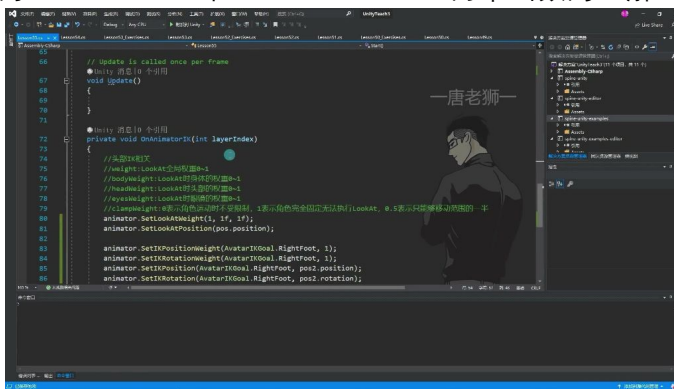


- 应用情景:
 - 拾取物品: 当角色需要抓取场景中的物品时, 使用IK可以让手部精确贴合物品位置
 - 武器瞄准: 如持枪或弓箭时, 通过IK控制上半身旋转瞄准目标, 同时保持下半身正常行走
 - 示例: 生化危机游戏中架枪动作, 抬枪后开启IK控制, 通过鼠标旋转控制上半身瞄准
- 核心优势:
 - 实现局部肢体控制: 可以单独控制身体某部分(如上半身)而不影响其他部位运动
 - 自然动作衔接: IK能自动计算关节旋转, 使动作过渡更自然
 - 精确位置匹配: 确保肢体末端(如手部)能准确到达目标位置

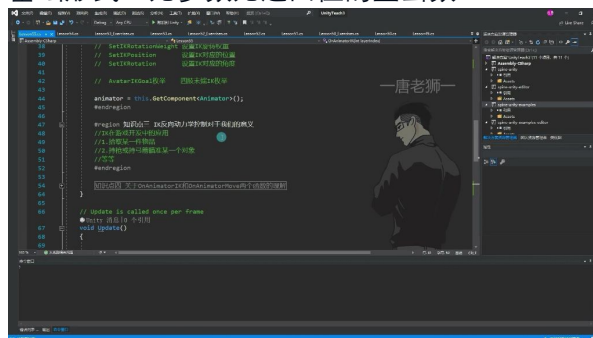


- 原理对比：
 - 正向动力学：父关节运动带动子关节（如肩膀转动带动手臂）
 - 反向动力学(IK)：子关节末端位置改变带动父关节旋转（如移动杯子带动手臂调整）

4. 关于OnAnimatorIK和OnAnimatorMove两个函数的理解 23:14



- 调用时机：
 - 在Update之后、LateUpdate之前调用
 - 每帧状态机和动画处理完成后才调用
 - 调用顺序：OnAnimatorIK → OnAnimatorMove
- 功能分工：
 - **OnAnimatorIK**：
 - 专门处理IK运动相关逻辑
 - 包含所有IK控制API调用（如设置权重、位置、旋转等）
 - 示例代码：
 - **OnAnimatorMove**：
 - 主要处理根运动(Root Motion)修改
 - 当动画本身包含移动且需要额外调整时使用
 - 基础形式：无参数无返回值的空函数



- 关键API：
 - SetLookAtWeight(): 设置头部IK全局权重及各部位权重（身体/头部/眼睛）

- SetIKPositionWeight()/SetIKRotationWeight(): 设置四肢IK位置/旋转权重
- SetIKPosition()/SetIKRotation(): 设置IK目标位置和角度
- AvatarIKGoal枚举: 指定要控制的肢体末端 (LeftHand/RightHand等)
- 使用注意:
 - 必须先Animator中开启IK通道
 - 权重值范围0-1, 0表示无影响, 1表示完全控制
 - clampWeight参数控制运动限制程度 (0无限制, 1完全固定)

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
IK (反向动力学) 基础概念	正向动力学 (子骨骼受父骨骼驱动) 与反向动力学 (子骨骼末端驱动父骨骼) 的区别, 人体动作举例 (如抬手vs拿杯子)	正向与反向动力学的本质区别 (父子骨骼驱动关系)	★★
Unity中IK控制步骤	1. 开启状态机层级IK Pass; 2. 在OnAnimatorIK回调函数中调用API; 3. 分头部与四肢控制 (权重+位置/角度设置)	IK Pass未开启导致代码失效	★★★
头部IK控制API	SetLookAtWeight (全局/身体/头部/眼睛权重); SetLookAtPosition (目标点位置)	权重参数含义 (如clampWeight限制运动范围)	★★
四肢IK控制API	SetIKPositionWeight+SetIKPosition (位置控制); SetIKRotationWeight+SetIKRotation (角度控制)	四肢枚举选择 (AvatarIKGoal.RightHand等); 位置与角度权重的独立作用	★★★
IK应用场景	1. 拾取物品; 2. 持枪瞄准 (上半身IK+下半身自由移动); 3. 动态注视目标	IK与动画融合的优先级问题	★★★
特殊生命周期函数	OnAnimatorIK: IK逻辑处理; OnAnimatorMove: 根运动修改; 调用时机: 动画处理完成后, LateUpdate前	与Update中处理动画的区别 (避免冲突)	★★